The Config Service might become a single point of failure in your system. If all the applications rely on it to fetch configuration data, you need to ensure it's highly available. The first step toward that goal is deploying multiple instances of Config Service in a production environment. If one of them stops working for some reason, another replica can provide the required configuration. In chapter 7, you'll learn more about scaling applications and how to make that work in Kubernetes.

However, scaling the Config Service is not enough. Since it's using a remote Git repository as the configuration data backend, you'll need to make that interaction more resilient too. First, you can define a timeout to prevent the config server from waiting too long to establish a connection with the remote repository. You can do so with the `spring.cloud.config.server.git.timeout` property.

Spring Cloud Config is implemented to clone the remote repository locally upon the first request for configuration data. I recommend using the `spring.cloud.config .server.git.clone-on-start` property so that the repo clone happens at startup. Even though it makes the startup phase a bit slower, it makes your deployment fail faster if there's any difficulty communicating with the remote repository, rather than waiting for the first request to find out that something is wrong. Also, it makes the first request from a client faster.

The local copy of the repository improves the config server's fault tolerance because it ensures it can return configuration data to the client applications even if the communication with the remote repository is temporarily failing (for example, if GitHub is down or there's a problem with the network). However, if the config server hasn't cloned the repository locally yet, there's no fallback in place. That's why it's better to fail fast at startup and investigate the issue right away.